

First-Order Logic

One task we logicians are interested in is that of analyzing the notion of “proof” — to make it as rigorous as any other notion in mathematics.

— Raymond Smullyan
The Lady or the Tiger?, 1982

This chapter extends the machinery of propositional logic to **first-order logic (FOL)**, also called both **predicate logic** and the first-order **predicate calculus**. While first-order logic enjoys a degree of expressiveness that makes it suitable for reasoning about computation, it does not admit completely automated reasoning.

FOL extends PL with predicates, functions, and quantifiers. As in our discussion of PL, we first introduce the syntax of FOL and its semantics. We then build on the semantic argument method of PL to provide a method of proving first-order validity.

Section 2.6 reviews *decidability* and *complexity*. A decidable problem has an algorithm, which is a procedure that always finishes with a correct answer on every instance of the problem. While validity of PL formulae is decidable, validity of FOL formulae is not. Complexity is the study of the intrinsic hardness of a decidable problem.

The optional Section 2.7 proves the soundness and completeness of the semantic argument method. It also presents two classic theorems that are applied in Chapter 10.

2.1 Syntax

All formulae of PL evaluate to **true** or **false**. FOL is not so simple. In FOL, **terms** evaluate to values other than truth values such as integers, people, or cards of a deck. However, we are getting ahead of ourselves: just as in PL,

the syntax of FOL is independent of its meaning. The most basic terms are **variables** $x, y, z, x_1, x_2, \dots$ and **constants** $a, b, c, a_1, a_2, \dots$.

More complicated terms are constructed using **functions**. An n -ary function f takes n terms as arguments. Notationally, we represent generic FOL functions by symbols $f, g, h, f_1, f_2, \dots$. A constant can also be viewed as a 0-ary function.

Example 2.1. The following are all terms:

- a , a constant (or 0-ary function);
- x , a variable;
- $f(a)$, a unary function f applied to a constant;
- $g(x, b)$, a binary function g applied to a variable x and a constant b ;
- $f(g(x, f(b)))$.

■

The propositional variables of PL are generalized to **predicates**, denoted $p, q, r, p_1, p_2, \dots$. An n -ary predicate takes n terms as arguments. A **FOL propositional variable** is a 0-ary predicate, which we write $P, Q, R, P_1, P_2, \dots$.

Countably infinitely many constant, function, and predicate symbols are available.

An **atom** is $\top$, $\perp$, or an n -ary predicate applied to n terms. A **literal** is an atom or its negation.

Example 2.2. The following are all literals:

- P , a propositional variable (or 0-ary predicate);
- $p(f(x), g(x, f(x)))$, a binary predicate applied to two terms;
- $\neg p(f(x), g(x, f(x)))$.

The first two literals are also atoms.

■

A **FOL formula** is a literal, the application of a logical connective $\neg, \wedge, \vee, \rightarrow$, or $\leftrightarrow$ to a formula or formulae, or the application of a **quantifier** to a formula. There are two FOL quantifiers:

- the **existential quantifier** $\exists x. F[x]$, read “there exists an x such that $F[x]$ ”;
- and the **universal quantifier** $\forall x. F[x]$, read “for all x , $F[x]$ ”.

In $\forall x. F[x]$, x is the **quantified variable**, and $F[x]$ is the **scope** of the quantifier $\forall x$. For convenience, we sometimes refer informally to the scope of the quantifier $\forall x$ as the scope of the quantified variable x itself. The case is similar for $\exists x. F[x]$. Also, x in $F[x]$ is **bound** (by the quantifier). By convention, the period in “ $F[x]$ ” indicates that the scope of the quantified variable x extends as far as possible. We often abbreviate $\forall x. \forall y. F[x, y]$ by $\forall x, y. F[x, y]$.

Example 2.3. In

$$\underbrace{\forall x. p(f(x), x) \rightarrow (\exists y. \underbrace{p(f(g(x, y)), g(x, y))}_G \wedge q(x, f(x)))}_F ,$$

the scope of x is F , and the scope of y is G . This formula is read: “for all x , if $p(f(x), x)$ then there exists a y such that $p(f(g(x, y)), g(x, y))$ and $q(x, f(x))$ ”. ■

A variable is **free** in formula $F[x]$ if there is an occurrence of x that is not bound by any quantifier. Denote by $\text{free}(F)$ the set of free variables of a formula F . A variable is **bound** in formula $F[x]$ if there is an occurrence of x in the scope of a binding quantifier $\forall x$ or $\exists x$. Denote by $\text{bound}(F)$ the set of bound variables of a formula F . In general, it is possible that $\text{free}(F) \cap \text{bound}(F) \neq \emptyset$, as a variable x can have both free and bound occurrences.

A formula F is **closed** if it does not contain any free variables.

Example 2.4. In

$$F : \forall x. p(f(x), y) \rightarrow \forall y. p(f(x), y) ,$$

x only occurs bound, while y appears both free (in the antecedent) and bound (in the consequent). Thus, $\text{free}(F) = \{y\}$ and $\text{bound}(F) = \{x, y\}$. ■

If $\text{free}(F) = \{x_1, \dots, x_n\}$, then its **universal closure** is

$$\forall x_1. \dots \forall x_n. F ,$$

and its **existential closure** is

$$\exists x_1. \dots \exists x_n. F .$$

We usually write the universal and existential closures as $\forall * . F$ and $\exists * . F$, respectively.

The **subformulae** of a FOL formula are defined according to an extension of the PL definition of subformula:

- the only subformula of $p(t_1, \dots, t_n)$, where the t_i are terms, is $p(t_1, \dots, t_n)$;
- the subformulae of $\neg F$ are $\neg F$ and the subformulae of F ;
- the subformulae of $F_1 \wedge F_2$, $F_1 \vee F_2$, $F_1 \rightarrow F_2$, $F_1 \leftrightarrow F_2$ are the formula itself and the subformulae of F_1 and F_2 ;
- and the subformulae of $\exists x. F$ and $\forall x. F$ are the formula itself and the subformulae of F .

The **strict subformulae** of a formula excludes the formula itself.

The **subterms** of a FOL term are defined as follows:

- the only subterm of constant a or variable x is a or x itself, respectively;

- and the subterms of $f(t_1, \dots, t_n)$ are the term itself and the subterms of $t_1, \dots, t_n$.

The **strict subterms** of a term excludes the term itself.

Example 2.5. In

$$F : \forall x. p(f(x), y) \rightarrow \forall y. p(f(x), y) ,$$

the subformulae of F are

$$F , p(f(x), y) \rightarrow \forall y. p(f(x), y) , \forall y. p(f(x), y) , p(f(x), y) .$$

The subterms of $g(f(x), f(h(f(x))))$ are

$$g(f(x), f(h(f(x)))) , f(x) , f(h(f(x))) , h(f(x)) , x .$$

$f(x)$ occurs twice in $g(f(x), f(h(f(x))))$. ■

Example 2.6. Before discussing the formal semantics for FOL, we suggest translations of English sentences into FOL. The names of the constants, functions, and predicates are chosen to provide some intuition for the meaning of the FOL formulae.

- Every dog has its day.

$$\forall x. \text{dog}(x) \rightarrow \exists y. \text{day}(y) \wedge \text{itsDay}(x, y)$$

- Some dogs have more days than others.

$$\exists x, y. \text{dog}(x) \wedge \text{dog}(y) \wedge \#days(x) > \#days(y)$$

- All cats have more days than dogs.

$$\forall x, y. \text{dog}(x) \wedge \text{cat}(y) \rightarrow \#days(y) > \#days(x)$$

- Fido is a dog. Furrball is a cat. Fido has fewer days than does Furrball.

$$\text{dog}(\text{Fido}) \wedge \text{cat}(\text{Furrball}) \wedge \#days(\text{Fido}) < \#days(\text{Furrball})$$

- The length of one side of a triangle is less than the sum of the lengths of the other two sides.

$$\forall x, y, z. \text{triangle}(x, y, z) \rightarrow \text{length}(x) < \text{length}(y) + \text{length}(z)$$

- Fermat's Last Theorem.

$$\begin{aligned} & \forall n. \text{integer}(n) \wedge n > 2 \\ & \rightarrow \forall x, y, z. \\ & \quad \text{integer}(x) \wedge \text{integer}(y) \wedge \text{integer}(z) \wedge x > 0 \wedge y > 0 \wedge z > 0 \\ & \quad \rightarrow x^n + y^n \neq z^n \end{aligned}$$
■

2.2 Semantics

Having defined the syntax of FOL, we now define its **semantics**. Formulae of FOL evaluate to the truth values **true** and **false** as in PL. However, terms of FOL formulae evaluate to values from a specified domain. We extend the concept of interpretations to this more complex setting and then define the semantics of FOL in terms of interpretations.

First, we define a FOL **interpretation** I . The **domain** D_I of an interpretation I is a nonempty set of values or objects, such as integers, real numbers, dogs, people, or merely abstract objects. $|D_I|$ denotes the **cardinality**, or size, of D_I . Domains can be finite, such as the 52 cards of a deck of cards; countably infinite, such as the integers; or uncountably infinite, such as the reals. But all domains are nonempty.

The **assignment** α_I of interpretation I maps constant, function, and predicate symbols to elements, functions, and predicates over D_I . It also maps variables to elements of D_I :

- each variable symbol x is assigned a value x_I from D_I ;
- each n -ary function symbol f is assigned an n -ary function

$$f_I : D_I^n \rightarrow D_I$$

that maps n elements of D_I to an element of D_I ;

- each n -ary predicate symbol p is assigned an n -ary predicate

$$p_I : D_I^n \rightarrow \{\text{true}, \text{false}\}$$

that maps n elements of D_I to a truth value.

In particular, each constant (0-ary function symbol) is assigned a value from D_I , and each propositional variable (0-ary predicate symbol) is assigned a truth value.

An interpretation $I : (D_I, \alpha_I)$ is thus a pair consisting of a domain and an assignment.

Example 2.7. The formula

$$F : x + y > z \rightarrow y > z - x$$

contains the binary function symbols $+$ and $-$, the binary predicate symbol $>$, and the variables x , y , and z . Again, $+$, $-$, and $>$ are just symbols: we choose these names to provide intuition for the intended meaning of the formulae. We could just as easily have written

$$F' : p(f(x, y), z) \rightarrow p(y, g(z, x)) .$$

We construct a “standard” interpretation. The domain is the integers, $\mathbb{Z}$:

$$D_I = \mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\} .$$

To $+$ and $-$ we assign standard addition $+\mathbb{Z}$ and subtraction $-\mathbb{Z}$ of integers, respectively. To $>$ we assign the standard greater-than relation $>\mathbb{Z}$ of integers. Finally, to x , y , and z , we assign the values 13, 42, and 1, respectively. We ignore the countably infinitely many other constant, function, and predicate symbols that do not appear in F . We thus have interpretation $I : (\mathbb{Z}, \alpha_I)$, where

$$\alpha_I : \{+ \mapsto +\mathbb{Z}, - \mapsto -\mathbb{Z}, > \mapsto >\mathbb{Z}, x \mapsto 13, y \mapsto 42, z \mapsto 1, \dots\}.$$

The elision reminds us that, as always, α_I provides values for the countably infinitely many other constant, function, and predicate symbols. Usually, we do not write the elision. ■

Given a FOL formula F and interpretation $I : (D_I, \alpha_I)$, we want to compute if F evaluates to **true** under interpretation I , $I \models F$, or if F evaluates to **false** under interpretation I , $I \not\models F$. We define the semantics inductively as in PL. To start, define the meaning of truth symbols:

$$\begin{aligned} I &\models \top \\ I &\not\models \perp \end{aligned}$$

Next, consider more complicated atoms. α_I gives meaning $\alpha_I[x]$, $\alpha_I[c]$, and $\alpha_I[f]$ to variables x , constants c , and functions f . Evaluate arbitrary terms recursively:

$$\alpha_I[f(t_1, \dots, t_n)] = \alpha_I[f](\alpha_I[t_1], \dots, \alpha_I[t_n]) ,$$

for terms $t_1, \dots, t_n$. That is, define the value of $f(t_1, \dots, t_n)$ under α_I by evaluating the function $\alpha_I[f]$ over the terms $\alpha_I[t_1]$, ..., $\alpha_I[t_n]$. Similarly, evaluate arbitrary atoms recursively:

$$\alpha_I[p(t_1, \dots, t_n)] = \alpha_I[p](\alpha_I[t_1], \dots, \alpha_I[t_n]) .$$

Then

$$I \models p(t_1, \dots, t_n) \quad \text{iff} \quad \alpha_I[p(t_1, \dots, t_n)] = \text{true}$$

Having completed the base cases of our inductive definition, we turn to the inductive step. Assume that formulae F_1 and F_2 have fixed truth values. From these formulae, evaluate the semantics of more complex formulae. The logical connectives are handled in FOL in precisely the same way as in PL:

$$\begin{aligned} I &\models \neg F && \text{iff } I \not\models F \\ I &\models F_1 \wedge F_2 && \text{iff } I \models F_1 \text{ and } I \models F_2 \\ I &\models F_1 \vee F_2 && \text{iff } I \models F_1 \text{ or } I \models F_2 \\ I &\models F_1 \rightarrow F_2 && \text{iff, if } I \models F_1 \text{ then } I \models F_2 \\ I &\models F_1 \leftrightarrow F_2 && \text{iff } I \models F_1 \text{ and } I \models F_2, \text{ or } I \not\models F_1 \text{ and } I \not\models F_2 \end{aligned}$$

Example 2.8. Recall the formula

$$F : x + y > z \rightarrow y > z - x$$

of Example 2.7 and the interpretation $I : (\mathbb{Z}, \alpha_I)$, where

$$\alpha_I : \{+ \mapsto +_{\mathbb{Z}}, - \mapsto -_{\mathbb{Z}}, > \mapsto >_{\mathbb{Z}}, x \mapsto 13_{\mathbb{Z}}, y \mapsto 42_{\mathbb{Z}}, z \mapsto 1_{\mathbb{Z}}\}.$$

Compute the truth value of F under I as follows:

1. $I \models x + y > z$ since $\alpha_I[x + y > z] = 13_{\mathbb{Z}} +_{\mathbb{Z}} 42_{\mathbb{Z}} >_{\mathbb{Z}} 1_{\mathbb{Z}}$
2. $I \models y > z - x$ since $\alpha_I[y > z - x] = 42_{\mathbb{Z}} >_{\mathbb{Z}} 1_{\mathbb{Z}} -_{\mathbb{Z}} 13_{\mathbb{Z}}$
3. $I \models F$ by 1, 2, and the semantics of $\rightarrow$

■

For the quantifiers, let x be a variable. Define an x -**variant** of an interpretation $I : (D_I, \alpha_I)$ as an interpretation $J : (D_J, \alpha_J)$ such that

- $D_I = D_J$;
- and $\alpha_I[y] = \alpha_J[y]$ for all constant, free variable, function, and predicate symbols y , except possibly x .

That is, I and J agree on everything except possibly the value of variable x . Denote by $J : I \triangleleft \{x \mapsto v\}$ the x -variant of I in which $\alpha_J[x] = v$ for some $v \in D_I$. Then

$$\begin{aligned} I \models \forall x. F & \quad \text{iff for all } v \in D_I, I \triangleleft \{x \mapsto v\} \models F \\ I \models \exists x. F & \quad \text{iff there exists } v \in D_I \text{ such that } I \triangleleft \{x \mapsto v\} \models F \end{aligned}$$

In words, I is an interpretation of $\forall x. F$ iff all x -variants of I are interpretations of F . I is an interpretation of $\exists x. F$ iff some x -variant of I is an interpretation of F .

Example 2.9. Consider the formula

$$F : \exists x. f(x) = g(x)$$

and the interpretation $I : (D : \{\circ, \bullet\}, \alpha_I)$ in which

$$\alpha_I : \{f(\circ) \mapsto \circ, f(\bullet) \mapsto \bullet, g(\circ) \mapsto \bullet, g(\bullet) \mapsto \circ\}.$$

Compute the truth value of F under I as follows:

1. $I \triangleleft \{x \mapsto v\} \not\models f(x) = g(x)$ for $v \in D$
2. $I \not\models \exists x. f(x) = g(x)$ since $v \in D$ is arbitrary

In the first line, basic reasoning about the interpretation I reveals that f and g always disagree. The second line follows from the first by the semantics of existential quantification. ■

2.3 Satisfiability and Validity

A formula F is said to be **satisfiable** iff there exists an interpretation I such that $I \models F$. A formula F is said to be **valid** iff for all interpretations I , $I \models F$. Determining satisfiability and validity of formulae are important tasks in FOL. Recall that satisfiability and validity are dual: F is valid iff $\neg F$ is unsatisfiable.

Technically, satisfiability and validity only apply to closed FOL formulae, which do not have free variables. However, let us agree on a convention: if we say that a formula F such that $\text{free}(F) \neq \emptyset$ is valid, we mean that its universal closure $\forall *. F$ is valid; and if we say that it is satisfiable, we mean that its existential closure $\exists *. F$ is satisfiable. Duality still holds: a formula F with free variables is valid ($\forall *. F$ is valid) iff its negation is unsatisfiable ($\exists *. \neg F$ is unsatisfiable). Henceforth, we freely discuss the validity and satisfiability of formulae with free variables.

For arguing the validity of FOL formulae, we extend the semantic argument method from PL to FOL. Most of the concepts carry over to the FOL case without change. In addition to the rules for the logical connectives of PL (see Section 1.3), we have the following rules for the quantifiers.

- According to the semantics of universal quantification, from $I \models \forall x. F$, deduce $I \triangleleft \{x \mapsto v\} \models F$ for any $v \in D_I$.

$$\frac{I \models \forall x. F}{I \triangleleft \{x \mapsto v\} \models F} \quad \text{for any } v \in D_I$$

In practice, we usually apply this rule using a domain element v that was introduced earlier in the proof.

- Similarly, from the semantics of existential quantification, from $I \not\models \exists x. F$, deduce $I \triangleleft \{x \mapsto v\} \not\models F$ for any $v \in D_I$.

$$\frac{I \not\models \exists x. F}{I \triangleleft \{x \mapsto v\} \not\models F} \quad \text{for any } v \in D_I$$

Again, we usually apply this rule using a domain element v that was introduced earlier in the proof.

- According to the semantics of existential quantification, from $I \models \exists x. F$, deduce $I \triangleleft \{x \mapsto v\} \models F$ for some $v \in D_I$ that has *not* been previously used in the proof.

$$\frac{I \models \exists x. F}{I \triangleleft \{x \mapsto v\} \models F} \quad \text{for a fresh } v \in D_I$$

- Similarly, from the semantics of universal quantification, from $I \not\models \forall x. F$, deduce $I \triangleleft \{x \mapsto v\} \not\models F$ for some $v \in D_I$ that has *not* been previously used in the proof.

$$\frac{I \not\models \forall x. F}{I \triangleleft \{x \mapsto v\} \not\models F} \quad \text{for a fresh } v \in D_I$$

The restriction in the latter two rules corresponds to our intuition: if all we know is that $\exists x. F$, then we certainly do not know which value in particular satisfies F . Hence, we choose a new value v that does not appear previously in the proof: it was never introduced before by a quantification rule. Moreover, α_I does not already assign it to some constant, $\alpha_I[a]$, or to some function application, $\alpha_I[f(t_1, \dots, t_n)]$.

Notice the similarity between the first two and between the final two rules. The first two rules handle a case that is universal in character. Consider the second rule: if there does not exist an x such that F , then for all values, F does not hold. The final two rules are existential in character.

Lastly, the contradiction rule is modified for the FOL case.

- A contradiction exists if two variants of the original interpretation I disagree on the truth value of an n -ary predicate p for a given tuple of domain values.

$$\frac{J : I \triangleleft \dots \models p(s_1, \dots, s_n) \quad K : I \triangleleft \dots \not\models p(t_1, \dots, t_n) \quad \text{for } i \in \{1, \dots, n\}, \alpha_J[s_i] = \alpha_K[t_i]}{I \models \perp}$$

The intuition behind the contradiction rule is the following. The variants J and K are constructed only through the rules for quantification. Hence, the truth value of p on the given tuple of domain values is already established by I . Therefore, the disagreement between J and K on the truth value of p indicates a problem with I .

None of these rules cause branching, but several of the rules for the logical connectives do. Thus, a proof in general is a tree. A branch is closed if it contains a contradiction according to the (first-order) contradiction rule; it is open otherwise. All branches are closed in a finished proof of a valid formula. We exhibit the proof method through several examples.

Example 2.10. We prove that

$$F : (\forall x. p(x)) \rightarrow (\forall y. p(y))$$

is valid. Suppose not; then there is an interpretation I such that $I \not\models F$:

- | | | |
|----|--|---|
| 1. | $I \not\models F$ | assumption |
| 2. | $I \models \forall x. p(x)$ | 1 and semantics of $\rightarrow$ |
| 3. | $I \not\models \forall y. p(y)$ | 1 and semantics of $\rightarrow$ |
| 4. | $I \triangleleft \{y \mapsto v\} \not\models p(y)$ | 3 and semantics of $\forall$, for some $v \in D_I$ |
| 5. | $I \triangleleft \{x \mapsto v\} \models p(x)$ | 2 and semantics of $\forall$ |

Lines 2 and 3 state the case in which line 1 holds: the antecedent and consequent of F are respectively true and false under I . Line 4 states that because of 3, there must be a value $v \in D_I$ such that $I \triangleleft \{y \mapsto v\} \not\models p(y)$. Line 5 uses this same value v and the semantics of $\forall$ with 2 to derive a contradiction: under I , $p(v)$ is false by 4 and true by 5. Thus, F is valid. ■

For concision we shorten, for example, “semantics of $\forall$ ” to “ $\forall$ ” in the explanation column of our arguments.

Example 2.11. Consider the following relation between universal and existential quantification:

$$F : (\forall x. p(x)) \leftrightarrow (\neg \exists x. \neg p(x)) .$$

Is it valid? Suppose not. Then there is an interpretation I such that $I \not\models F$. Consider the forward ($\rightarrow$) and backward ($\leftarrow$) directions of $\leftrightarrow$ as separate cases. In the first case,

- | | | |
|----|---|--|
| 1. | $I \models \forall x. p(x)$ | assumption |
| 2. | $I \not\models \neg \exists x. \neg p(x)$ | assumption |
| 3. | $I \models \exists x. \neg p(x)$ | 2 and $\neg$ |
| 4. | $I \triangleleft \{x \mapsto v\} \models \neg p(x)$ | 3 and $\exists$, for some $v \in D_I$ |
| 5. | $I \triangleleft \{x \mapsto v\} \models p(x)$ | 1 and $\forall$ |

Lines 4 and 5 are contradictory. In line 5, we use the value introduced in line 4 with the semantics of $\forall$ and line 1. We are allowed to choose this same value v precisely because line 1 states that *for all* x , $p(x)$.

For the second case,

- | | | |
|----|---|--|
| 1. | $I \not\models \forall x. p(x)$ | assumption |
| 2. | $I \models \neg \exists x. \neg p(x)$ | assumption |
| 3. | $I \triangleleft \{x \mapsto v\} \not\models p(x)$ | 1 and $\forall$, for some $v \in D_I$ |
| 4. | $I \not\models \exists x. \neg p(x)$ | 2 and $\neg$ |
| 5. | $I \triangleleft \{x \mapsto v\} \not\models \neg p(x)$ | 4 and $\exists$ |
| 6. | $I \triangleleft \{x \mapsto v\} \models p(x)$ | 5 and $\neg$ |

Lines 3 and 6 are contradictory. Line 4 says that $\exists x. \neg p(x)$ is *false* under I . Thus, by the semantics of $\exists$, no value w from D_I is such that $p(w)$ is *true*. In particular, line 5 identifies v , introduced in line 3.

As both cases end in contradictions for arbitrary interpretation I , F is valid. ■

It is sometimes useful to reference known values, as the following simple example illustrates.

Example 2.12. To prove that

$$F : p(a) \rightarrow \exists x. p(x)$$

is valid, assume otherwise and derive a contradiction.

- | | | |
|----|--|---------------------|
| 1. | $I \not\models F$ | assumption |
| 2. | $I \models p(a)$ | 1 and $\rightarrow$ |
| 3. | $I \not\models \exists x. p(x)$ | 1 and $\rightarrow$ |
| 4. | $I \triangleleft \{x \mapsto \alpha_I[a]\} \not\models p(x)$ | 3 and $\exists$ |
| 5. | $I \models \perp$ | 2, 4 |

In line 4, we used the value assigned to a to instantiate the quantified variable of line 3, which has universal character. Because lines 2 and 4 are contradictory, F is valid. ■

To show that a formula F is invalid, it suffices to find an interpretation I such that $I \models \neg F$.

Example 2.13. Consider the formula

$$F : (\forall x. p(x, x)) \rightarrow (\exists x. \forall y. p(x, y)) .$$

To show that it is invalid, we find an interpretation I such that

$$I \models \neg((\forall x. p(x, x)) \rightarrow (\exists x. \forall y. p(x, y))) ,$$

or, according to the semantics of $\rightarrow$,

$$I \models (\forall x. p(x, x)) \wedge \neg(\exists x. \forall y. p(x, y)) .$$

Choose

$$D_I = \{0, 1\}$$

and

$$p_I = \{(0, 0), (1, 1)\} .$$

We use a common notation for defining relations: $p_I(a, b)$ is **true** iff $(a, b) \in p_I$. Here, $p_I(0, 0)$ is **true**, and $p_I(1, 0)$ is **false**.

Both $\forall x. p(x, x)$ and $\neg(\exists x. \forall y. p(x, y))$ evaluate to **true** under I , so

$$I \models (\forall x. p(x, x)) \wedge \neg(\exists x. \forall y. p(x, y)) ,$$

which shows that F is invalid. Interpretation I is a falsifying interpretation of F . ■

We apply the semantic argument method to more examples in Section 3.1.

Equivalence ($F_1 \Leftrightarrow F_2$) and implication ($F_1 \Rightarrow F_2$) extend directly from PL to FOL. Equivalence of and implication between two formulae can be argued using the semantic argument method. See, for example, Example 2.11.

2.4 Substitution

Substitution for FOL is more complex than substitution for PL because of quantification. We introduce two types of substitution in this section with the goal of generalizing Propositions 1.15 and 1.17 to the FOL setting. As in PL, substitution allows us to consider the validity of entire sets of formulae simultaneously.

First, we define the **renaming** of a quantified variable. If variable x is quantified in F so that F has the form $F[\forall x. G[x]]$, then the renaming of x to fresh variable x' produces the formula $F[\forall x'. G[x']]$. A “fresh variable” is any variable that does not occur in F . By the semantics of universal quantification, the original and final formulae are equivalent. The case is similar for existential quantification. Often, we simply say that a variable is renamed to mean that its bound occurrences are renamed to a fresh variable. Free occurrences of variables are never renamed.

Example 2.14. Renaming the bound variable x to fresh variable x' in

$$F : p(x) \wedge \forall x. q(x, y)$$

produces

$$F' : p(x) \wedge \forall x'. q(x', y) .$$

Renaming y does not cause any change because y does not occur bound. ■

A **substitution** is a map from FOL formulae to FOL formulae:

$$\sigma : \{F_1 \mapsto G_1, \dots, F_n \mapsto G_n\} .$$

As in PL, $\text{domain}(\sigma) = \{F_1, \dots, F_n\}$ and $\text{range}(\sigma) = \{G_1, \dots, G_n\}$. To compute the application of σ to F , $F\sigma$, replace each occurrence of F_i in F by G_i simultaneously. When both subformulae F_j and F_k are in the domain of σ and F_k is a strict subformula of F_j , replace occurrences of F_j by G_j .

Example 2.15. Consider formula

$$F : (\forall x. p(x, y)) \rightarrow q(f(y), x)$$

and substitution

$$\sigma : \{x \mapsto g(x), y \mapsto f(x), q(f(y), x) \mapsto \exists x. h(x, y)\} .$$

Then

$$F\sigma : (\forall x. p(g(x), f(x))) \rightarrow \exists x. h(x, y) .$$

Notice how there are more bound occurrences of x in $F\sigma$ than in F . ■

Use care when substitutions include quantifiers in the domain. Substituting for a quantified subformula $\forall x. F$ requires that all of $\forall x. F$ be replaced.

Example 2.16. Consider formula

$$F : \exists y. p(x, y) \wedge p(y, x)$$

and substitution

$$\sigma : \{\exists y. p(x, y) \mapsto p(x, a)\} ,$$

where a is a constant. $F\sigma = F$ because the scope of the quantifier $\exists y$ in F is $p(x, y) \wedge p(y, x)$, not just $p(x, y)$. ■

2.4.1 Safe Substitution

A restricted application of substitution has a useful semantic property.

Define for a substitution σ its set of free variables:

$$V_\sigma = \bigcup_i (\text{free}(F_i) \cup \text{free}(G_i)) .$$

V_σ consists of the free variables of all formulae F_i and G_i of the domain and range of σ . Compute the **safe substitution** $F\sigma$ of formula F as follows:

1. For each quantified variable x in F such that $x \in V_\sigma$, rename x to a fresh variable to produce F' .
2. Compute $F'\sigma$.

Example 2.17. Consider again formula

$$F : (\forall x. p(x, y)) \rightarrow q(f(y), x)$$

and substitution

$$\sigma : \{x \mapsto g(x), y \mapsto f(x), q(f(y), x) \mapsto \exists x. h(x, y)\} .$$

To compute the safe substitution $F\sigma$, first compute

$$\begin{aligned} V_\sigma &= \text{free}(x) \cup \text{free}(g(x)) \cup \text{free}(y) \cup \text{free}(f(x)) \\ &\quad \cup \text{free}(q(f(y), x)) \cup \text{free}(\exists x. h(x, y)) \\ &= \{x\} \cup \{x\} \cup \{y\} \cup \{x\} \cup \{x, y\} \cup \{y\} \\ &= \{x, y\} \end{aligned}$$

Then

1. $x \in V_\sigma$, so rename bound occurrences in F :

$$F' : (\forall x'. p(x', y)) \rightarrow q(f(y), x) .$$

x also occurs free in F .

2. $F'\sigma : (\forall x'. p(x', f(x))) \rightarrow \exists x. h(x, y)$.

■

The first step of computing a safe substitution becomes trivial if each quantified variable has a unique name.

Example 2.18. Consider formula

$$F : (\forall z. p(z, y)) \rightarrow q(f(y), x) ,$$

in which the quantified variable has a different name than any free variable of F or the substitution

$$\sigma : \{x \mapsto g(x), y \mapsto f(y), q(f(y), x) \mapsto \exists w. h(w, y)\} .$$

Compared to Example 2.17, the quantified variable z in F and the quantified variable w of σ have different names than any other variable of F or σ . The safe substitution is the unrestricted substitution

$$F\sigma : (\forall z. p(z, f(y))) \rightarrow \exists w. h(w, y) .$$

■

Proposition 2.19 (Substitution of Equivalent Formulae). *Consider substitution*

$$\sigma : \{F_1 \mapsto G_1, \dots, F_n \mapsto G_n\}$$

such that for each i , $F_i \Leftrightarrow G_i$. Then $F \Leftrightarrow F\sigma$ when $F\sigma$ is computed as a safe substitution.

The language of Propositions 2.19 and 1.15 are almost identical.

2.4.2 Schema Substitution

Example 2.11 proves an interesting relation between universal and existential quantification ($\forall x. p(x) \Leftrightarrow \neg \exists x. \neg p(x)$), but the result is not general. We would like to prove that for any FOL formula F ,

$$H : (\forall x. F) \Leftrightarrow (\neg \exists x. \neg F)$$

is valid. H is a **formula schema** (plural: **schemata**). Formula schema and **schema substitutions** provide the desired generality.

A **formula schema** H contains at least one **placeholder** $F_1, F_2, \dots$. For example, F is a placeholder in the formula schema H above. A formula schema can also have **side conditions** that specify that certain variables do not occur free in the placeholders.

Consider a substitution σ mapping placeholders to FOL formulae. A **schema substitution** is an (unrestricted) application of σ to a formula schema. It is legal only if σ obeys the side conditions of the formula schema.

Example 2.20. Recall from Example 2.11 that

$$(\forall x. p(x)) \Leftrightarrow (\neg \exists x. \neg p(x))$$

is valid. It can act as a formula schema. First, rewrite the formula using placeholders:

$$H : (\forall x. F) \Leftrightarrow (\neg \exists x. \neg F) .$$

H does not have any side conditions. Next, to prove the validity of

$$G : (\forall x. \exists y. q(x, y)) \leftrightarrow (\neg \exists x. \neg \exists y. q(x, y)) ,$$

show that G is derivable from H via a schema substitution:

$$\sigma : \{F \mapsto \exists y. q(x, y)\} .$$

Then $H\sigma = G$ ($H\sigma$ is syntactically identical to G), so that by Proposition 2.25 below, G is valid. ■

A formula schema H is **valid** if $H\sigma$ is valid for every schema substitution σ that obeys the side conditions of H . Apply the semantic method to prove the validity of a formula schema.

Example 2.21. To prove the validity of the formula schema

$$H : (\forall x. F_1 \wedge F_2) \leftrightarrow (\forall x. F_1) \wedge (\forall x. F_2) ,$$

consider the two directions. First, assume that $I \not\models (\forall x. F_1 \wedge F_2) \rightarrow (\forall x. F_1) \wedge (\forall x. F_2)$:

- | | | |
|----|--|------------|
| 1. | $I \models \forall x. F_1 \wedge F_2$ | assumption |
| 2. | $I \not\models (\forall x. F_1) \wedge (\forall x. F_2)$ | assumption |
| 3. | $I \models (\exists x. \neg F_1) \vee (\exists x. \neg F_2)$ | 2, $\neg$ |

There are two cases to consider:

- | | | |
|-----|--|--------------------------------------|
| 4a. | $I \models \exists x. \neg F_1$ | 3, $\vee$ |
| 5a. | $I \triangleleft \{x \mapsto v\} \models \neg F_1$ | 4a, $\exists$, for some $v \in D_I$ |
| 6a. | $I \triangleleft \{x \mapsto v\} \models F_1 \wedge F_2$ | 1, $\forall$ |
| 7a. | $I \triangleleft \{x \mapsto v\} \models F_1$ | 6a, $\wedge$ |

ending in a contradiction. The second disjunctive case is similar.

For the second main case, assume that $I \not\models (\forall x. F_1) \wedge (\forall x. F_2) \rightarrow (\forall x. F_1 \wedge F_2)$:

- | | | |
|----|--|-------------------------------------|
| 1. | $I \not\models \forall x. F_1 \wedge F_2$ | assumption |
| 2. | $I \models (\forall x. F_1) \wedge (\forall x. F_2)$ | assumption |
| 3. | $I \models \exists x. \neg F_1 \vee \neg F_2$ | 1, $\neg$ |
| 4. | $I \triangleleft \{x \mapsto v\} \models \neg F_1 \vee \neg F_2$ | 3, $\exists$, for some $v \in D_I$ |
| 5. | $I \models \forall x. F_1$ | 2, $\wedge$ |
| 6. | $I \models \forall x. F_2$ | 2, $\wedge$ |
| 7. | $I \triangleleft \{x \mapsto v\} \models F_1$ | 5, $\forall$ |
| 8. | $I \triangleleft \{x \mapsto v\} \models F_2$ | 6, $\forall$ |

Again, there are two cases to consider:

- | | | |
|-----|--|-----------|
| 9a. | $I \triangleleft \{x \mapsto v\} \models \neg F_1$ | 4, $\vee$ |
|-----|--|-----------|

ending in a contradiction. The second disjunctive case is similar. Thus, H is a valid formula schema. ■

We now consider formula schemata with side conditions.

Example 2.22. Consider the formula schema with side condition

$$H : (\forall x. F) \leftrightarrow F \text{ provided } x \notin \text{free}(F) .$$

If we disregard the side condition, then H is an invalid formula schema as, for example,

$$G_1 : (\forall x. p(x)) \leftrightarrow p(x) ,$$

obtained from H by schema substitution

$$\sigma : \{F \mapsto p(x)\} ,$$

is invalid. However, σ is disallowed by the side condition that x should not occur free in F .

H (with the side condition) is a valid formula schema. Thus, the formula

$$G_2 : (\forall x. \exists y. p(z, y)) \leftrightarrow \exists y. p(z, y)$$

is valid: obtain it via the schema substitution

$$\sigma : \{F \mapsto \exists y. p(z, y)\} ,$$

which obeys H 's side condition. ■

Reasoning about the validity of a formula schema with side conditions usually requires invoking its side conditions during a semantic argument.

Example 2.23. To prove the validity of

$$H : (\forall x. F) \leftrightarrow F \text{ provided } x \notin \text{free}(F) ,$$

consider the two directions of $\leftrightarrow$. First,

1. $I \models \forall x. F$ assumption
2. $I \not\models F$ assumption
3. $I \models F$ 1, $\forall$, since $x \notin \text{free}(F)$
4. $I \models \perp$ 2, 3

Second,

1. $I \not\models \forall x. F$ assumption
2. $I \models F$ assumption
3. $I \models \exists x. \neg F$ 1 and $\neg$
4. $I \models \neg F$ 3, $\exists$, since $x \notin \text{free}(F)$
5. $I \models \perp$ 2, 4

Thus, H is a valid formula schema. ■

Example 2.24. To prove the validity of

$$H : (\forall x. F_1 \wedge F_2) \leftrightarrow (\forall x. F_1) \wedge F_2 \quad \text{provided } x \notin \text{free}(F_2) ,$$

consider two cases. First,

- | | |
|---|--|
| 1. $I \models \forall x. F_1 \wedge F_2$ | assumption |
| 2. $I \not\models (\forall x. F_1) \wedge F_2$ | assumption |
| 3. $I \models (\forall x. F_1) \wedge (\forall x. F_2)$ | 1, valid schema from Example 2.21 |
| 4. $I \models (\forall x. F_1) \wedge F_2$ | 3, Example 2.23, since $x \notin \text{free}(F_2)$ |
| 5. $I \models \perp$ | 2, 4 |

Observe the application of valid formula schemata from previous examples in lines 3 and 4. Second,

- | | |
|---|--|
| 1. $I \not\models \forall x. F_1 \wedge F_2$ | assumption |
| 2. $I \models (\forall x. F_1) \wedge F_2$ | assumption |
| 3. $I \models (\forall x. F_1) \wedge (\forall x. F_2)$ | 2, Example 2.23, since $x \notin \text{free}(F_2)$ |
| 4. $I \models \forall x. F_1 \wedge F_2$ | 3, Example 2.21 |
| 5. $I \models \perp$ | 1, 4 |

Thus, H is a valid formula schema. ■

Proposition 2.25 (Formula Schema). *If H is a valid formula schema and σ is a substitution obeying H 's side conditions, then $H\sigma$ is also valid.*

The valid PL formula

$$(P \rightarrow Q) \leftrightarrow (\neg P \vee Q)$$

can be treated as a valid formula schema:

$$(F_1 \rightarrow F_2) \leftrightarrow (\neg F_1 \vee F_2) .$$

In general, valid propositional templates are valid formulae schemata, so that Proposition 2.25 generalizes Proposition 1.17.

2.5 Normal Forms

The normal forms of PL extend to FOL. A FOL formula F can be transformed into negation normal form (NNF) by using the procedure of Section 1.6 augmented with these two (schema) equivalences:

$$\neg \forall x. F[x] \Leftrightarrow \exists x. \neg F[x] \quad \neg \exists x. F[x] \Leftrightarrow \forall x. \neg F[x] .$$

Example 2.26. We apply the procedure to find a formula in NNF that is equivalent to

$$G : \forall x. (\exists y. p(x, y) \wedge p(x, z)) \rightarrow \exists w. p(x, w) .$$

Each formula below is equivalent to G and is obtained from the previous one through an application of an equivalence.

1. $\forall x. (\exists y. p(x, y) \wedge p(x, z)) \rightarrow \exists w. p(x, w)$
2. $\forall x. \neg(\exists y. p(x, y) \wedge p(x, z)) \vee \exists w. p(x, w)$
3. $\forall x. (\forall y. \neg(p(x, y) \wedge p(x, z))) \vee \exists w. p(x, w)$
4. $\forall x. (\forall y. \neg p(x, y) \vee \neg p(x, z)) \vee \exists w. p(x, w)$

Formula 2 follows from the equivalence

$$F_1 \rightarrow F_2 \Leftrightarrow \neg F_1 \vee F_2 .$$

Formula 3 arises from an application of

$$\neg \exists x. F[x] \Leftrightarrow \forall x. \neg F[x] ,$$

and the final formula, which is in NNF, follows from De Morgan's Law. ■

A formula is in **prenex normal form (PNF)** if all of its quantifiers appear at the beginning of the formula:

$$Q_1 x_1. \dots Q_n x_n. F[x_1, \dots, x_n] ,$$

where $Q_i \in \{\forall, \exists\}$ and F is quantifier-free. Every FOL formula F can be transformed into an equivalent formula F' in PNF. To compute an equivalent PNF F' of F ,

1. Convert F into NNF formula F_1 .
2. When multiple quantified variables have the same name, rename them to fresh variables, resulting in F_2 .
3. Remove all quantifiers from F_2 to produce quantifier-free formula F_3 .
4. Add the quantifiers before F_3 ,

$$F_4 : Q_1 x_1. \dots Q_n x_n. F_3 ,$$

where the Q_i are the quantifiers such that if Q_j is in the scope of Q_i in F_1 , then $i < j$.

F_4 is equivalent to F .

Example 2.27. We apply the procedure to find a PNF equivalent of

$$F : \forall x. \neg(\exists y. p(x, y) \wedge p(x, z)) \vee \exists y. p(x, y) .$$

1. Write F in NNF:

$$F_1 : \forall x. (\forall y. \neg p(x, y) \vee \neg p(x, z)) \vee \exists y. p(x, y) .$$

2. Rename quantified variables:

$$F_2 : \forall x. (\forall y. \neg p(x, y) \vee \neg p(x, z)) \vee \exists w. p(x, w) .$$

3. Remove all quantifiers to produce quantifier-free formula

$$F_3 : \neg p(x, y) \vee \neg p(x, z) \vee p(x, w) .$$

4. Add the quantifiers before F_3 :

$$F_4 : \forall x. \forall y. \exists w. \neg p(x, y) \vee \neg p(x, z) \vee p(x, w) .$$

Alternately, choose order

$$F'_4 : \forall x. \exists w. \forall y. \neg p(x, y) \vee \neg p(x, z) \vee p(x, w) .$$

Both F_4 and F'_4 are equivalent to F . However,

$$G : \forall y. \exists w. \forall x. \neg p(x, y) \vee \neg p(x, z) \vee p(x, w)$$

is not equivalent to F .

■

A FOL formula is in CNF (DNF) if it is in PNF and its main quantifier-free subformula is in CNF (DNF). CNF and DNF equivalents are obtained by transforming formula F into PNF formula F' and then applying the relevant procedure of Section 1.6 to the main quantifier-free subformula of F' .

2.6 Decidability and Complexity

We review the main concepts from decidability and complexity theory. **Bibliographic Remarks** refers the interested reader to other texts that focus on these topics.

2.6.1 Satisfiability as a Formal Language

Satisfiability of formulae is the primary decision problem in logic. We can formalize satisfiability as a formal language decision problem. Consider PL. Let L_{PL} be the set of all satisfiable formulae. That is, the word $w \in L_{\text{PL}}$ iff

1. w is a syntactically well-formed formulae: it parses according to the definition of Section 1.1;
2. and when w is viewed as a PL formula F , F is satisfiable.

Then the formal decision problem is the following: given a word w , is $w \in L_{\text{PL}}$?

Satisfiability of FOL formulae can be similarly formalized as a language question. Let L_{FOL} be the set of all FOL formulae (words that parse according to Section 2.1) that are satisfiable. Then the formal decision problem is the following: given a word w , is $w \in L_{\text{FOL}}$? In other words, is it a well-formed FOL formula, and if so, is it satisfiable? Dually, we can define the validity problem for PL and FOL.

2.6.2 Decidability

A formal language L is **decidable** if there exists a procedure that, given a word w , (1) eventually halts and (2) answers *yes* if $w \in L$ and *no* if $w \notin L$. Other terms for “decidable” are **recursive** and **Turing-decidable**. A procedure for a decidable language is called an **algorithm**. Satisfiability of PL formulae is decidable: the truth-table method is a decision procedure.

A formal language is **undecidable** if it is not decidable.

A formal language L is **semi-decidable** if there exists a procedure that, given a word w , (1) halts and answers *yes* iff $w \in L$, (2) halts and answers *no* if $w \notin L$, or (3) does not halt if $w \notin L$. The possible outcomes (2) and (3) for the case $w \notin L$ mean that the procedure may or may not halt. Unlike a decidable language, the procedure is only guaranteed to halt if $w \in L$. Other terms for “semi-decidable” are **partially decidable**, **recursively enumerable**, and **Turing-recognizable**.

The terms “Turing-decidable” and “Turing-recognizable” arise from Alan Turing’s classic formalization of procedures as **Turing machines**. A Turing machine consists of a finite automaton coupled with an infinite tape and tape head. Each cell of the tape can hold one character from a finite alphabet. The state of the automaton changes based on its control structure and the character currently under the tape head. During a state change, the automaton instructs the tape head to write a new character to the current cell and to move one position left or right.

Church and Turing showed that L_{FOL} is undecidable: there does not exist an algorithm for deciding if a FOL formula F is satisfiable (and similarly for validity). However, there is a procedure that halts and says *yes* if F is valid, so validity is semi-decidable. We describe such a procedure based on the semantic argument method in Section 2.7.

2.6.3 ★Complexity

If a language is decidable, then one considers the complexity of the decision problem. We define several of the main complexity classes here. A language L is **polynomial-time decidable**, or in PTIME (also, in P), if there exists a procedure that, given w , answers *yes* when $w \in L$, answers *no* when $w \notin L$, and halts with the answer in a number of steps that is at most proportionate to some polynomial of the size of w . For example, determining if the word w is a well-formed FOL formula is polynomial-time decidable and can be implemented using standard parsing methods.

A language L is **nondeterministic-polynomial-time decidable**, or in NPTIME (also, in NP), if there exists a nondeterministic procedure that, given w ,

1. guesses a **witness** W to the fact that $w \in L$ that is at most proportionate in size to some polynomial of the size of w ;

2. checks in time at most proportionate to some polynomial of the size of w that W really is a witness to $w \in L$;
3. and answers *yes* if the check succeeds and *no* otherwise.

For example, L_{PL} is in NP, as exhibited by the following nondeterministic procedure for deciding if a PL formula is satisfiable:

1. parse the input w as formula F (return *no* if w is not a well-formed PL formula);
2. guess an interpretation I , which is linear in the size of w ;
3. check that $I \models F$.

I is the witness to the satisfiability of F .

A language L is in **co-NP** if its complement language $\bar{L}$ is in NP. For example, unsatisfiability of PL formulae is in **co-NP** because satisfiability is in NP. It is not known if unsatisfiability of PL formulae is in NP. While a satisfiable PL formula has a polynomial size witness of its satisfiability (a satisfying interpretation I), there is no known polynomial size witness of unsatisfiability.

A language L is **NP-hard** if every instance $v \in L'$ of every other NP decidable language L' can be reduced to deciding an instance $w_{L'}^v \in L$. Moreover, the size of $w_{L'}^v$ must be at most proportionate to some polynomial of the size of v . That is, L is NP-hard if every query $v \in L'$ of every NP language L' can be encoded into a query $w \in L$, where w is not much larger than v . L is **NP-complete** if it is in NP and is NP-hard.

L_{PL} is NP-complete. Indeed, L_{PL} , also called SAT, was the first language shown to be NP-complete. We proved that L_{PL} is in NP above by describing a nondeterministic polynomial time procedure. The Cook-Levin theorem shows that all NP-languages L can be reduced to L_{PL} , so that L_{PL} is NP-hard. They exhibited a polynomial time algorithm that, given L and input w , constructs an encoding into PL of a simulation of a run of a nondeterministic Turing machine for L on w . The encoding as PL formula F has length that is polynomial in the length of w . F is satisfiable iff the Turing machine decides that $w \in L$.

For discussing the complexity of decision problems and algorithms, we use the standard notation. Consider a function $f(n)$ over integers. For example, $f(n) = \log n$, $f(n) = n^2$, $f(n) = 2^n$, or $f(n) = 2^{2^n}$. A function $g(n)$ is of at most **order** $f(n)$ if there exist a scalar $c \geq 0$ and an integer $n_0 \geq 0$ such that

$$\forall n \geq n_0. g(n) \leq cf(n) .$$

$O(f(n))$ denotes the set of all functions of at most order $f(n)$. Similarly, $\Omega(f(n))$ denotes the set of all function of at least order $f(n)$: a function $g(n)$ is of at least order $f(n)$ if there exist a scalar $c \geq 0$ and an integer $n_0 \geq 0$ such that

$$\forall n \geq n_0. g(n) \geq cf(n) .$$

Finally, $\Theta(f(n)) = \Omega(f(n)) \cap O(f(n))$ denotes the set of all functions of precisely order $f(n)$.

Example 2.28.

- $3n^2 + n \in O(n^2)$
- $3n^2 + n \in \Omega(n^2)$
- $3n^2 + n \in \Theta(n^2)$
- $\frac{1}{99}n^2 + n \in \Omega(n^2)$
- $3n^2 + n \in O(2^n)$
- $3n^2 + n \in \Omega(n)$
- $3n^2 + n \notin \Omega(2^n)$
- $3n^2 + n \notin \Theta(2^n)$
- $2^n \in \Omega(n^3)$
- $2^n \notin O(n^3)$

■

A decision problem has time complexity $O(f(n))$ if there exists a decision algorithm P for the problem and a function $g(n) \in O(f(n))$ such that P runs in time at most $g(n)$ on input of size n . A decision problem has time complexity $\Omega(f(n))$ if there exists a function $g(n) \in \Omega(f(n))$ such that all decision algorithms P for the problem run in time at least $g(n)$ on input of size n . Finally, a decision problem has time complexity $\Theta(f(n))$ if it has time complexities $\Omega(f(n))$ and $O(f(n))$.

Example 2.29. The algorithm SAT for deciding PL satisfiability runs in time $\Theta(2^n)$, where n is the number of variables in the input formula, because each level of recursion branches. Hence, the problem of PL satisfiability has time complexity $O(2^n)$. ■

2.7 ★Meta-Theorems of First-Order Logic

We prove that the semantic argument method for FOL is sound and, given a proper strategy of applying the proof rules, complete. A proof method is **sound** if every formula that has a proof according to the method is valid. A proof method is **complete** if every formula that is valid has a proof according to the method. That the semantic argument method is sound means that a closed semantic argument for $I \models F$ proves the validity of F ; and that the semantic argument method is complete means that every valid formula F of FOL has a closed semantic argument proving its validity. Because there exists a complete proof method for FOL, FOL is a complete logic: every valid formula of FOL has a proof of its validity.

The second half of this section is devoted to proving two classic theorems that we apply in Chapter 10.

2.7.1 Simplifying the Language of FOL

In preparation for the proofs, we simplify the language of FOL without losing expressiveness. Exercises 1.3 and 4.6 show that we have many redundant logical connectives. We choose to use only the logical constant $\top$ and the connectives $\neg$ and $\wedge$, from which the others can be constructed. Additionally, we need only one quantifier since $\exists x. F$ is equivalent to $\neg\forall x. \neg F$. We choose $\forall$.

A second simplification is more involved. The goal is to remove constant and function symbols from the language by using predicate symbols instead. Given a formula F , let S be the set of function symbols appearing in it. Associate with each n -ary function symbol f of S a new $(n+1)$ -ary predicate p_f . Then for each occurrence of a function f in a literal L of F

$$L[f(t_1, \dots, t_n)] ,$$

replace L in F with the new formula

$$\exists x. p_f(t_1, \dots, t_n, x) \wedge L[x] .$$

After all replacements, the resulting formula G does not contain any function symbols.

The next step ensures that the new predicate p_f describes a function f : it associates with each tuple of domain values $v_1, \dots, v_n$ precisely one value v . For each introduced predicate p_f , construct the formula

$$I_f : \forall \bar{x}. \exists y. p_f(\bar{x}, y) \wedge (\forall z. p_f(\bar{x}, z) \rightarrow y = z) .$$

Then construct the formula

$$H : \left(\bigwedge_{f \in S} I_f \right) \rightarrow G .$$

The equality predicate $=$ is not yet defined. To make $=$ an equivalence relation, assert that it is reflexive, symmetric, and transitive:

$$\begin{aligned} E : & (\forall x. x = x) \\ & \wedge (\forall x, y. x = y \rightarrow y = x) \\ & \wedge (\forall x, y, z. x = y \wedge y = z \rightarrow x = z) \end{aligned}$$

Additionally, every predicate symbol p appearing in G should obey $=$:

$$E_p : \forall \bar{x}, \bar{y}. \bar{x} = \bar{y} \rightarrow (p(\bar{x}) \leftrightarrow p(\bar{y}))$$

Let T be the set of predicate symbols of G . Construct the final formula

$$F' : \left(E \wedge \bigwedge_{p \in T} E_p \right) \rightarrow H .$$

F' is valid iff F is valid. Moreover, F' does not contain any function symbols.

For the special case of constant symbols, it is simpler to replace $F[a]$ with $F' : \forall x. F[x]$.

For the remainder of this section, we consider a version of FOL with only the logical constant $\top$, the connectives $\neg$ and $\wedge$, the quantifier $\forall$, and predicate symbols. It is equivalent in expressive power to the richer language studied earlier in the chapter.

2.7.2 Semantic Argument Proof Rules

In this simplified form of FOL, we need only the following seven proof rules.

- For handling negation:

$$\frac{I \models \neg F}{I \not\models F} \quad \frac{I \not\models \neg F}{I \models F}$$

- For handling conjunction:

$$\frac{I \models F \wedge G}{I \models F} \quad \frac{I \not\models F \wedge G}{I \not\models F \mid I \not\models G}$$

$$I \models G$$

- For handling universal quantification:

$$\frac{I \models \forall x. F}{I \triangleleft \{x \mapsto v\} \models F} \quad \text{for any } v \in D_I$$

and

$$\frac{I \not\models \forall x. F}{I \triangleleft \{x \mapsto v\} \not\models F} \quad \text{for a fresh } v \in D_I$$

- For deriving a contradiction:

$$\frac{J : I \triangleleft \dots \models p(x_1, \dots, x_n)}{K : I \triangleleft \dots \not\models p(y_1, \dots, y_n)} \quad \text{for } i \in \{1, \dots, n\}, \alpha_J[x_i] = \alpha_K[y_i]$$

$$I \models \perp$$

Only the second rule for conjunction requires a case analysis.

2.7.3 Soundness and Completeness

That the semantic argument method is sound is fairly obvious for the first six rules: each follows almost directly from the semantics of FOL. The final rule for deriving a contradiction requires some explanation. The variants J and K are constructed only through the rules for handling quantification so that they simply assign values to the arguments of p . Hence, the truth value of p on the given tuple of domain values is already established by I . Furthermore, the disagreement between J and K on the truth value of p indicates that I is not in fact an interpretation. Therefore, we have the following theorem.

Theorem 2.30 (Sound). *If every branch of a semantic argument proof of $I \not\models F$ closes, then F is valid.*

Completeness is more complicated. We want to show that there exists a closed semantic argument proof of $I \not\models F$ when F is valid. Our strategy is as follows. We define a procedure for applying the proof rules. When applying the quantification rules, the procedure selects values from a predetermined countably infinite domain. We then show that when some falsifying interpretation I exists (such that $I \not\models F$) our procedure constructs, *at the limit*, a falsifying interpretation. Therefore, F must be valid if the procedure actually discovers an argument in which all branches are closed. We now proceed according to this proof plan.

Let D be a countably infinite domain of values $v_1, v_2, v_3, \dots$ which we can enumerate in some fixed order. Start the semantic argument by placing $I \not\models F$ at the root and marking it as *unused*. Now assume that the procedure has constructed a partial semantic argument and that each line is marked as either *used* or *unused*. We describe the next iteration.

Select the earliest line $L : I \models G$ or $L : I \not\models G$ in the argument that is marked *unused*, and choose the appropriate proof rule to apply according to the root symbol of G 's parse tree. To apply a rule, add the appropriate deductions at the end of every open branch that passes through line L ; mark each new deduction as *unused*; and mark L as *used*. The application of the negation rules and the first conjunction rule is then straightforward. Applying the second (branching) conjunction rule introduces a fork at the end of every open branch, doubling the number of open branches. In applying the second quantification rule, choose the next domain element v_i that does not appear in the semantic argument so far. For the first quantification rule, assume that G has the form $\forall x. H$. Choose the first value v_i on which $\forall x. H$ has not been instantiated in any ancestor of L . Additionally, consider $I \models G$ as a second “deduction” of this rule (so that both $I \triangleleft \{x \mapsto v_i\} \models H$ and $I \models G$ are added to every branch passing through L and marked as *unused*). This trick guarantees that x of $\forall x. H$ is instantiated on every domain element without preventing the rest of the proof from progressing. Finally, close any branch that has a contradiction resulting from a deduction in this iteration.

Recall that a semantic argument is finished if no further applications of rules are possible. In our proof procedure, this situation occurs when all lines are marked as *used*. Although we can never construct a finished semantic argument with infinitely many lines in practice, we can reason about such arguments. For example, such an argument has an infinitely long branch. For suppose not: then every branch has finite length, so there must be an infinite number of these finite branches. But such a situation requires a deduction step that results in an infinite number of branches, whereas each proof rule produces at most two branches. This result is known as **König's Lemma**. We next prove that each open branch of a finished semantic argument describes a falsifying interpretation.

Lemma 2.31. *Each open branch of a finished semantic argument for $I \not\models F$ defines a falsifying interpretation of F .*

Proof. We apply structural induction on the formulae appearing in the branch to conclude that each line $L : I \models G$ or $L : I \not\models G$ holds, including $I \not\models F$. In that case, I is a falsifying interpretation of F . The technique of **structural induction** is defined in Section 4.4.

For the base case, consider lines in which G is an atom. As the contradiction rule is never applied (otherwise, the branch would be closed) no contradiction exists. Therefore, each instance $I \triangleleft \dots \models p(x_1, \dots, x_n)$ or $I \triangleleft \dots \not\models p(x_1, \dots, x_n)$ defines the truth value of p on one tuple of domain elements without contradicting any other definition on the branch. (For tuples of domain elements on which p is not explicitly defined on the branch, p may take any value, say false.)

Consider when G is formed by applying a logical connective to one or two formulae. As the procedure applied the appropriate proof rule, the inductive hypothesis and the semantics of the logical connectives tell us that L holds. For example, consider the case $L : I \models F_1 \wedge F_2$. Then $I \models F_1$ and $I \models F_2$ appear on the branch, and both lines hold by the inductive hypothesis. The reader may verify the other logical connectives with similar reasoning.

Consider the case $L : I \not\models \forall x. F$. For L to hold, it must be the case that for some fresh domain value $\mathbf{v}$, $M : I \triangleleft \{x \mapsto \mathbf{v}\} \not\models F$. But the procedure guarantees that M is a descendant of L . Moreover, F is a subformula of $\forall x. F$, so the inductive hypothesis asserts that M holds. Then so does L .

Consider the case $L : I \models \forall x. F$. For L to hold, it must be the case that for all domain values $\mathbf{v}$, $M : I \triangleleft \{x \mapsto \mathbf{v}\} \models F$. But the procedure guarantees that such a line exists for every $\mathbf{v}$. Moreover, F is a subformula of $\forall x. F$, so the inductive hypothesis asserts that each such line holds. Hence, so does L , finishing the proof. ■

Remark 2.32. The formulae that appear on an open branch of a finished semantic proof comprise a **Hintikka set**. The proof strategy that we employed is essentially that used in proving **Hintikka's Lemma**, which asserts that a Hintikka set is satisfiable.

Remark 2.33. We defined the procedure with a fixed countably infinite domain in mind and then proved that an open branch of a finished semantic argument corresponds to at least one falsifying interpretation. Therefore, we have proved an additional fact: every satisfiable FOL formula is satisfied by an interpretation with a countable domain. This result is **Löwenheim's Theorem**.

Theorem 2.34 (Complete). *The semantic argument method is complete: each valid formula F has a semantic argument proof (in which every branch is closed).*

Proof. Suppose that F is valid, yet no semantic argument proof exists. Then a finished semantic argument constructed according to our procedure has an open branch. By Lemma 2.31, this branch describes a falsifying interpretation of F , a contradiction. Hence, all branches of a finished semantic argument must in fact be closed (and thus finite). By König's Lemma, the semantic argument itself has finite size. ■

2.7.4 Additional Theorems

Is a countable (but possibly infinite) set S of satisfiable formulae **simultaneously satisfiable**? That is, does there exist a single interpretation I that satisfies every member of S ? The **Compactness Theorem** relates simultaneous satisfiability of S to satisfiability of the conjunction of each finite subset of S . Dually, we might consider whether the disjunction of a countable set of formulae is valid.

Theorem 2.35 (Compactness Theorem). *A countable set of first-order formulae S is simultaneously satisfiable iff the conjunction of every finite subset is satisfiable.*

Proof. The forward direction is clear: if I simultaneously satisfies the members of S , then it satisfies each finite conjunction.

For the other direction, extend the proof procedure of the previous section as follows. Arrange the members of S in some order $F_1, F_2, F_3, \dots$, which is possible because S is countable. Start the procedure with $I \not\models \neg F_1$. At the end of each iteration of the procedure, choose the next formula F_i in the sequence and append $I \not\models \neg F_i$ to every open branch, marking it as *unused*. Since each finite subset of S is satisfiable, at least one branch remains open and the procedure does not terminate.

A finished semantic argument constructed in this manner enumerates an interpretation that falsifies every $\neg F_i$ and thus satisfies every F_i . Hence, S is simultaneously satisfiable. ■

Remark 2.36. This proof proves an additional fact that extends Löwenheim's Theorem: every simultaneously satisfiable countable set of FOL formulae is simultaneously satisfied by an interpretation with a countable domain. This result is the **Löwenheim-Skolem Theorem**.

We apply the next theorem, the **Craig Interpolation Lemma**, in Chapter 10. It asserts that if $F \rightarrow G$ is valid, then there is a formula H (called an **interpolant**) such that $F \rightarrow H$ and $H \rightarrow G$ are valid and whose predicates and free variables occur in both F and G . The proof is constructive: it describes a procedure for extracting the interpolant from a proof of the validity of $F \rightarrow G$. However, proofs constructed via the proof rules of Section 2.7.2 are not directly amenable to the interpolation procedure. We describe an alternate set of proof rules instead and show that any proof constructed from

the rules of 2.7.2 can be translated into a proof using the new rules. Then we prove the Craig Interpolation Lemma using these new proof rules.

One trick that will prove convenient is the following. Associate a fresh variable x_i with each domain value v_i introduced during the proof. Whenever a variant $I \triangleleft \{x \mapsto v_i\}$ is used, rename x to the variable x_i corresponding to the value v_i in both the variant interpretation and the formula. This renaming does not affect the soundness of the proof, but it makes contradictions more obvious.

The new rules are the following:

- For handling double negation:

$$\frac{I \models \neg\neg F}{I \models F} \qquad \frac{I \not\models \neg\neg F}{I \not\models F}$$

- For handling conjunction:

$$\frac{I \models F \wedge G}{I \models F} \qquad \frac{I \not\models F \wedge G}{I \not\models F \mid I \not\models G}$$

$$I \models G$$

and

$$\frac{I \models \neg(F \wedge G)}{I \models \neg F \mid I \models \neg G} \qquad \frac{I \not\models \neg(F \wedge G)}{I \not\models \neg F}$$

$$I \not\models \neg G$$

- For handling universal quantification:

$$\frac{I \models \forall x. F}{I \triangleleft \{x \mapsto v\} \models F} \qquad \frac{I \not\models \neg\forall x. F}{I \triangleleft \{x \mapsto v\} \not\models \neg F} \qquad \text{for any } v \in D_I$$

and

$$\frac{I \not\models \forall x. F}{I \triangleleft \{x \mapsto v\} \not\models F} \qquad \frac{I \models \neg\forall x. F}{I \triangleleft \{x \mapsto v\} \models \neg F} \qquad \text{for a fresh } v \in D_I$$

- For deriving a contradiction (recall our trick of renaming variables to correspond uniquely to domain values):

$$\frac{J : I \triangleleft \dots \models p(x_1, \dots, x_n) \quad K : I \triangleleft \dots \not\models p(x_1, \dots, x_n)}{I \models \perp}$$

and

$$\frac{J : I \triangleleft \dots \models p(x_1, \dots, x_n) \quad K : I \triangleleft \dots \models \neg p(x_1, \dots, x_n)}{I \models \perp} \qquad \frac{J : I \triangleleft \dots \not\models p(x_1, \dots, x_n) \quad K : I \triangleleft \dots \not\models \neg p(x_1, \dots, x_n)}{I \models \perp}$$

The important characteristic (for proving the interpolation lemma) of this set of proof rules is that premises and deductions agree on the use of $\models$ or $\not\models$, except in the contradiction rules. In contrast, the negation rules of Section 2.7.2 do not have this property. We obtained this property by folding each negation rule into every other rule.

Before proving the interpolation lemma, let us prove that the new semantic argument proof system based on these rules is sound and complete. Soundness is fairly obvious; for completeness, we briefly describe how to map a proof from the system of Section 2.7.2 to a proof using these rules.

Lemma 2.37. *Every proof in the proof system of Section 2.7.2 has a corresponding proof in the new proof system.*

Proof. In constructing the new proof, ignore any use of the negation rules of Section 2.7.2, instead choosing from the (doubled) set of conjunction and quantification rules depending on whether a $\neg$ is at the root of the parse tree of a formula. Use the new negation rules to remove double negations when necessary. For deriving a contradiction, one of the three cases represented by the contradiction rules must occur when a contradiction occurs in the original proof. ■

We can now prove the theorem.

Theorem 2.38 (Craig Interpolation Lemma). *If $F \rightarrow G$ is valid, then there exists a formula H such that $F \rightarrow H$ and $H \rightarrow G$ are valid and whose predicates and free variables occur in both F and G .*

Proof. We prove the result by describing a procedure that extracts from a (closed) semantic argument proof of the validity of $F \rightarrow G$ the interpolant H . For convenience, let the proof itself begin with the lines

1. $I \models F$ assumption
2. $I \not\models G$ assumption

Notice that with the new set of proof rules, only $\models$ rules will be applied to deductions stemming from line 1, while only $\not\models$ rules will be applied to those stemming from line 2. The three contradiction rules correspond to three possible situations: a contradiction between $I \models F$ and $I \not\models G$ ($F \rightarrow G$ is valid), within $I \models F$ itself (F is unsatisfiable), and within $I \not\models G$ itself (G is valid).

The procedure runs backwards through a proof. It associates with each line L of the proof a set of *positive* formulae U and a set of *negative* formulae V . U consists of formulae on lines from which L descends (including itself) that are satisfied by their interpretation (lines of the form $K \models F_1$). V consists of formulae on lines from which L descends (including itself) that are falsified by their interpretation (lines of the form $K \not\models F_2$). Define L 's **characteristic formula** as

$$\bigwedge U \rightarrow \bigvee V ,$$

written as $\{U\} \rightarrow \{V\}$ for concision. That the branch on which L lies ends in a contradiction implies that $\{U\} \rightarrow \{V\}$ is valid. The procedure constructs for each line an interpolant X of $\{U\} \rightarrow \{V\}$; that is, X is such that

$$\bigwedge U \Rightarrow X \quad \text{and} \quad X \Rightarrow \bigvee V$$

and the predicates and free variables of X appear in both U and V . The interpolant of line 2 of the proof is the interpolant H that we seek.

Let us begin with the end of a branch, $L : I \models \perp$. It must have been deduced via a contradiction. If the first contradiction rule produced L , then its characteristic formula is of the form

$$\{U, p(x_1, \dots, x_n), \perp\} \rightarrow \{V, p(x_1, \dots, x_n)\} ,$$

where the variable renaming trick ensures that the arguments to p are syntactically the same. Its parent has characteristic formula

$$\{U, p(x_1, \dots, x_n)\} \rightarrow \{V, p(x_1, \dots, x_n)\} ,$$

and both have interpolant $p(x_1, \dots, x_n)$. If the second contradiction rule produced L , then its characteristic formula is of the form

$$\{U, p(x_1, \dots, x_n), \neg p(x_1, \dots, x_n), \perp\} \rightarrow \{V\} ,$$

and its parent's is of the form

$$\{U, p(x_1, \dots, x_n), \neg p(x_1, \dots, x_n)\} \rightarrow \{V\} .$$

Both have interpolant $\perp$ ($\neg\top$ in the restricted language). Similarly, if the third contradiction rule produced L , then the interpolant is $\top$.

Consider lines derived via the conjunction rules. Suppose $L : I \models F$ is deduced from $I \models F \wedge G$. Then the characteristic formulae of L and its parent are

$$\{U, F \wedge G, F\} \rightarrow \{V\} \quad \text{and} \quad \{U, F \wedge G\} \rightarrow \{V\} ,$$

respectively. If L has interpolant X , then so does its parent. The case is similar for a line $L : I \not\models \neg F$ deduced from $I \not\models \neg(F \wedge G)$.

For the next conjunction rule, suppose that $L : I \not\models F$ is deduced on one branch from $I \not\models F \wedge G$. Then L is at a fork in the proof and has sibling line $L' : I \not\models G$. The characteristic formulae of L , L' , and their parent are

$$\{U\} \rightarrow \{V, F \wedge G, F\} , \quad \{U\} \rightarrow \{V, F \wedge G, G\} , \quad \text{and} \quad \{U\} \rightarrow \{V, F \wedge G\} ,$$

respectively. If L and L' have interpolants X and Y , respectively, then their parent has interpolant $X \wedge Y$.

Similarly, suppose $L : I \models \neg F$ is deduced on one branch from $I \models \neg(F \wedge G)$ (so that its sibling is $L' : I \models \neg G$). If L and L' have interpolants X and Y , respectively, then their parent has interpolant $X \vee Y$ ($\neg(\neg X \wedge \neg Y)$ in the restricted language).

The interpolant X of a line L derived via a double-negation rule passes directly to its parent, for the characteristic formula of L simply has a repetition $\neg\neg F$ of a formula F that the parent's characteristic formula does not have.

We turn to the quantification rules. Consider a line $L : I \triangleleft \{z \mapsto v\} \not\models F$ derived from $I \not\models \forall x. F$. L and its parent M have characteristic formulae

$$\{U\} \rightarrow \{V, \forall x. F, F\} \quad \text{and} \quad \{U\} \rightarrow \{V, \forall x. F\} ,$$

respectively. Moreover, v is fresh, and thus z does not appear in either U or V according to our trick. Hence, z cannot occur free in L 's interpolant X . It thus follows that

$$\forall *. \forall z. X \rightarrow V \vee \forall x. F \vee F$$

is equivalent to

$$\forall *. X \rightarrow V \vee \forall x. F \vee \forall z. F$$

and thus to

$$\forall *. X \rightarrow V \vee \forall x. F .$$

Therefore, X is an interpolant of M . Similarly, X is an interpolant of the parent of $L : I \triangleleft \{z \mapsto v\} \models \neg F$ deduced from $I \models \neg \forall x. F$.

Consider $L : I \triangleleft \{z \mapsto v\} \models F$ with interpolant X derived from $I \models \forall x. F$, where z is not necessarily fresh. The characteristic formulae of L and its parent M are

$$\{U, \forall x. F, F\} \rightarrow \{V\} \quad \text{and} \quad \{U, \forall x. F\} \rightarrow \{V\} ,$$

respectively. Clearly,

$$U \wedge \forall x. F \wedge F \Rightarrow X$$

implies that

$$U \wedge \forall x. F \Rightarrow X .$$

Hence, X is an interpolant of M when z is not free in U or V (so that z is not free in X) and when it is free in both. However, if z is free in V but not in U , then X is not an interpolant of M . But $\forall z. X$ is an interpolant. In particular, we have

$$U \wedge \forall x. F \Rightarrow \forall z. X$$

and

$$\forall z. X \Rightarrow V \quad \text{because} \quad \forall z. X \Rightarrow X \quad \text{and} \quad X \Rightarrow V .$$

For the final case, suppose that $L : I \triangleleft \{z \mapsto v\} \not\models \neg F$ is deduced from $I \not\models \neg \forall x. F$. The characteristic formula of L is

$$\{U\} \rightarrow \{V, \neg \forall x. F, F\} .$$

Then X is the interpolant of the parent M unless z is free in U but not free in V . In the latter case, the interpolant is $\exists z. X$ ($\neg \forall z. \neg X$ in the restricted language). The reasoning is similar to the previous case, completing the proof. ■

2.8 Summary

Building on the presentation of PL in Chapter 1, this chapter introduces first-order logic (FOL). It covers:

- Its *syntax*. How one constructs a FOL formula. Variables, terms, function symbols, predicate symbols, atoms, literals, logical connectives, quantifiers.
- Its *semantics*. What a FOL formula means. Truth values **true** and **false**. Interpretations: domain and assignments. Difference between a function (predicate) symbol and a function (predicate) over a domain.
- *Satisfiability* and *validity*. Whether a FOL formula evaluates to **true** under any or all interpretations. Semantic argument method.
- *Substitution*, which is a tool for manipulating formulae and making general claims. Safe and schema substitutions. Substitution of equivalent formulae. Valid schemata.
- *Normal forms*. A normal form is a set of syntactically restricted formulae such that every FOL formula is equivalent to some member of the set.
- A review of *decidability and complexity theory*, which provides the concepts necessary for discussing decidability and complexity questions in logic.
- *Meta-theorems*. Semantic argument method is sound and complete. Compactness Theorem. Craig Interpolation Lemma.

The results of Section 2.7 are the groundwork for our theoretical treatment of the Nelson-Oppen combination method in Chapter 10.

FOL is the most general logic that is discussed in this book. Its applications include software and hardware design and analysis, knowledge representation, and complexity and decidability theory.

FOL is a complete logic: every valid FOL formula has a proof in the semantic argument method. However, validity is undecidable. Many applications benefit from complete automation, which is impossible when considering all of FOL. Therefore, Chapter 3 introduces first-order theories, which formalize interesting structures, such as integers, rationals, lists, stacks, and arrays. Part II of this book explores algorithms for reasoning within these theories.

Bibliographic Remarks

For a complete and concise presentation of propositional and first-order logic, see Smullyan's text *First-Order Logic* [87]. The semantic argument method is similar to Smullyan's tableau method. Also, the proofs of completeness of the semantic argument method, the Compactness Theorem, and the Craig Interpolation Lemma are inspired by Smullyan's presentation.

The history of the development of mathematical logic is rich. For an overview, see [98] and related articles in *The Stanford Encyclopedia of Philosophy*. We mention in particular *Hilbert's program* of the 1920s — see, for example, [38] — to find a consistent and complete axiomatization of arithmetic. Gödel's two *incompleteness theorems* proved that such a goal is impossible. The first incompleteness theorem, which Gödel presented in a lecture in September, 1930, and then in [36], states that any axiomatization of arithmetic contains theorems that are not provable within the theory. The second, which Gödel had proved by October, 1930, states that a theory such as Peano arithmetic cannot prove its own consistency unless it is itself inconsistent. Earlier, Gödel proved that first-order logic is complete [35]: every theorem has a proof. However, Church — and, independently, Turing — proved that satisfiability in first-order logic is undecidable [13]. Thus, while every theorem of first-order logic has a finite proof, invalid formulae need not have a finite proof of their invalidity.

For an introduction to formal languages, decidability, and complexity theory, see [85, 72, 41].

Exercises

2.1 (English and FOL). Encode the following English sentences into FOL.

- (a) Some days are longer than others.
- (b) In all the world, there is but one place that I call home.
- (c) My mother's mother is my grandmother.
- (d) The intersection of two convex sets is convex.

2.2 (FOL validity & satisfiability). For each of the following FOL formulae, identify whether it is valid or not. If it is valid, prove it with a semantic argument; otherwise, identify a falsifying interpretation.

- (a) $(\forall x, y. p(x, y) \rightarrow p(y, x)) \rightarrow \forall z. p(z, z)$
- (b) $\forall x, y. p(x, y) \rightarrow p(y, x) \rightarrow \forall z. p(z, z)$
- (c) $(\exists x. p(x)) \rightarrow \forall y. p(y)$
- (d) $(\forall x. p(x)) \rightarrow \exists y. p(y)$
- (e) $\exists x, y. (p(x, y) \rightarrow (p(y, x) \rightarrow \forall z. p(z, z)))$

2.3 (Semantic argument). Use the semantic argument method to prove the following formula schemata.

- (a) $\neg(\forall x. F) \Leftrightarrow \exists x. \neg F$
- (b) $\neg(\exists x. F) \Leftrightarrow \forall x. \neg F$
- (c) $\forall x, y. F \Leftrightarrow \forall y, x. F$
- (d) $\exists y. \forall x. F \Rightarrow \forall x. \exists y. F$
- (e) $\exists x. F \vee G \Leftrightarrow (\exists x. F) \vee (\exists y. G)$
- (f) $\exists x. F \rightarrow G \Leftrightarrow (\forall x. F) \rightarrow (\exists x. G)$
- (g) $\exists x. F \vee G \Leftrightarrow (\exists x. F) \vee G$, provided $x \notin \text{free}(G)$
- (h) $\forall x. F \vee G \Leftrightarrow (\forall x. F) \vee G$, provided $x \notin \text{free}(G)$
- (i) $\exists x. F \wedge G \Leftrightarrow (\exists x. F) \wedge G$, provided $x \notin \text{free}(G)$
- (j) $\forall x. F \rightarrow G \Leftrightarrow (\exists x. F) \rightarrow G$, provided $x \notin \text{free}(G)$

2.4 (Normal forms). Put the following formulae into prenex normal form.

- (a) $(\forall x. \exists y. p(x, y)) \rightarrow \forall x. p(x, x)$
- (b) $\exists z. (\forall x. \exists y. p(x, y)) \rightarrow \forall x. p(x, z)$
- (c) $\forall w. \neg(\exists x, y. \forall z. p(x, z) \rightarrow q(y, z)) \wedge \exists z. p(w, z)$

2.5 (★Characteristic formula). Why is the characteristic formula of a line on a closed branch of a semantic argument valid?